



Modelling Room Acoustics with a GPU

Jacob Josiah Webber

18 August 2017

MSc in High Performance Computing

The University of Edinburgh

Year of Presentation: 2017

Abstract

This is the bit where you summarise what is in your thesis.

Contents

1	Introduction	1
2	Background and Literature Review	2
2.1	Room Acoustics	2
2.1.1	Classical Acoustics	2
2.1.2	Finite Difference Schemes	3
2.2	Graphics Processing Units (GPUs)	4
2.2.1	GPUs and parallelism	5
2.2.2	GPU Memory Hierarchies	5
2.3	The State of the Art in FDTD Acoustical Modelling	5
2.3.1	Memory Considerations	5
2.4	Other Methods	6
2.4.1	Ray Tracing	6
2.4.2	Spectral Methods	6
3	The Proposed Data Structure and Method	7
4	Design and Implementation	8
4.1	Design Overview	8
4.2	Software Development Process	8
4.3	Debugging and Profiling Tools	8
5	Results and Analysis	9
5.1	Some results	9
5.1.1	More results	9
6	Conclusions	10
6.1	Future Work	10
A	NVIDIA K20 Device Details	11
B	Stuff which no-one will read	12

List of Tables

5.1	This is a simple table. More complicated tables can have headings which pass over more than one column	9
-----	---	---

List of Figures

6.1 The University Crest	10
------------------------------------	----

Acknowledgements

This template is a slightly modified version of the one developed by Prof. Charles Duncan for MSc students in the Dept. of Meteorology. His acknowledgement follows:

This template has been produced with help from many former students who have shown different ways of doing things. Please make suggestions for further improvements.

Chapter 1

Introduction

“There is no competition of sounds between a nightingale and a violin.”

Dejan Stojanovic

The way we perceive sound depends on the environment in which we hear it as well as how the sound was created. Listening to a sound in a space like a church or a cathedral is very different to listening in a bedroom. Being able to predict how a space will affect a sound has practical use in the design of new spaces, particularly those spaces where acoustic performance is critical. A great deal of effort goes into the design of new concert halls. This would be greatly enhanced by accurate acoustic models of spaces. The aim then, of research in this area, is to provide an accurate prediction of how a space will sound after being given its physical properties.

The key technique currently used for these simulations is the finite-difference time-domain (FDTD) method for finding approximate solutions to the wave equation across a three dimensional space. This forms a simple stencil. This stencil process across a vast number of points in memory is well suited to the use of graphics processing units (GPUs) which have high band width graphics memory and can operate on very many points in a massively parallel fashion.

In the work leading up to this report, currently state of the art GPU acoustics techniques as outlined in [1] and [2] were recreated. In addition to this a novel technique was devised and implemented that performs these simulations. This approach was designed to make best use of graphics memory, which is relatively scarce.

This report includes some background on the theory of acoustics and difference schemes, a brief background in GPUs and an overview of the new techniques that were devised and trialled in the work leading up to this report. This is followed by description of the details of the implementation in CUDA C as well as software development, profiling and debugging strategies and tools. There is then a comparative analysis of existing and novel techniques both in terms of performance in compute time and memory use. The report finishes with some conclusions and ideas for further work.

Chapter 2

Background and Literature Review

“He who loves practice without theory is like the sailor who boards ship without a rudder and compass and never knows where he may cast.”

Leonardo da Vinci

2.1 Room Acoustics

2.1.1 Classical Acoustics

Sound is modelled using the wave equation:

$$\frac{\partial^2 P}{\partial x^2} - \frac{1}{c^2} \frac{\partial^2 P}{\partial t^2} = 0 \quad (2.1)$$

where scalar field P is the deviation from the ambient air pressure, and c is the speed of sound. This result is derived by, among others, Richard Feynman in [3]. It is trivial to generalise this for 3 dimensional systems, which yields

$$\nabla^2 P - \frac{1}{c^2} \frac{\partial^2 P}{\partial t^2} = 0 \quad (2.2)$$

where

$$\nabla \equiv \hat{\mathbf{i}} \frac{\partial}{\partial x} + \hat{\mathbf{j}} \frac{\partial}{\partial y} + \hat{\mathbf{k}} \frac{\partial}{\partial z} \quad (2.3)$$

Intuitively we know that a room’s acoustic properties are dependent on its size and what materials form the walls and other surfaces. This is reflected in the solutions to the wave equation, where almost all environmental data is encoded in boundary conditions

imposed on the P scalar field. In real world applications these boundary conditions are too sophisticated for the wave equation to be solved analytically. This sophistication stems from both the shape of the room and the reflective properties of its boundaries. These reflective properties are a function of the frequency of the sound reflected.

2.1.2 Finite Difference Schemes

Finite difference schemes are therefore used to solve the wave equation for complex room shapes. These can provide an approximate numerical solution to the continuous wave equation.

A simple equivalence between first and second order derivatives and their corresponding central difference schemes is shown in 2.4 and 2.5.

$$\frac{df(u)}{du} \approx \frac{f(u+h) - f(u-h)}{2h} \quad (2.4)$$

$$\frac{d^2f(u)}{du^2} \approx \frac{f(u+h) - 2f(u) + f(u-h)}{h^2} \quad (2.5)$$

This can be easily discretised if h is set to be the distance in space or time between samples.

When discretising the space within the room, the scalar field of pressure within room is represented as a three dimensional array of floating point numbers. The letters l , m , and p are chosen to represent the samples index in each spacial dimension within the array, with n giving the time step. This means $P_{l,m,p}^n$ gives an approximation to P at the point (x, y, z) at time t where $x = lH$, $y = mH$, $z = pH$ and $t = nT$. T and H are the sample rates in time and space respectively.

When equation 2.2 is expanded out, it is shown that in order to compute a numerical approximation of the wave equation, discrete equivalents are required to the derivatives in time, and in each of the three spacial dimensions.

$$\frac{\partial^2 P}{\partial t^2} = c^2 \left(\frac{\partial^2 P}{\partial x^2} + \frac{\partial^2 P}{\partial y^2} + \frac{\partial^2 P}{\partial z^2} \right) \quad (2.6)$$

To replace these continuous partial derivatives a new finite difference operator, δ , is introduced. From 2.5 the following relations can be derived by simply replacing h with H :

$$\delta_t^2 P_{l,m,p}^n \equiv \frac{1}{T^2} \left(P_{l,m,p}^{n+1} - 2P_{l,m,p}^n + P_{l,m,p}^{n-1} \right) \approx \frac{\partial^2 P}{\partial t^2} \quad (2.7)$$

$$\delta_x^2 P_{l,m,p}^n \equiv \frac{1}{H^2} \left(P_{l+1,m,p}^n - 2P_{l,m,p}^n + P_{l-1,m,p}^n \right) \approx \frac{\partial^2 P}{\partial x^2} \quad (2.8)$$

$$\delta_y^2 P_{l,m,p}^n \equiv \frac{1}{H^2} \left(P_{l,m+1,p}^n - 2P_{l,m,p}^n + P_{l,m-1,p}^n \right) \approx \frac{\partial^2 P}{\partial y^2} \quad (2.9)$$

$$\delta_z^2 P_{l,m,p}^n \equiv \frac{1}{H^2} \left(P_{l,m,p+1}^n - 2P_{l,m,p}^n + P_{l,m,p-1}^n \right) \approx \frac{\partial^2 P}{\partial z^2} \quad (2.10)$$

It is now possible to use these operators to discretise 2.6.

$$\delta_t^2 P_{l,m,p}^n = c^2 \left(\delta_x^2 P_{l,m,p}^n + \delta_y^2 P_{l,m,p}^n + \delta_z^2 P_{l,m,p}^n \right) \quad (2.11)$$

Equation 2.11 can then be expanded using the results of equations 2.7-2.10. This yields

$$P_{l,m,p}^{n+1} - 2P_{l,m,p}^n + P_{l,m,p}^{n-1} = \frac{c^2 T^2}{H^2} \left(P_{l+1,m,p}^n + P_{l-1,m,p}^n + P_{l,m+1,p}^n + P_{l,m-1,p}^n + P_{l,m,p+1}^n + P_{l,m,p-1}^n - 6P_{l,m,p}^n \right) \quad (2.12)$$

Letting the Courant number be defined as

$$\lambda \equiv \frac{cT}{H} \quad (2.13)$$

and our stencil

$$S \equiv P_{l+1,m,p}^n + P_{l-1,m,p}^n + P_{l,m+1,p}^n + P_{l,m-1,p}^n + P_{l,m,p+1}^n + P_{l,m,p-1}^n \quad (2.14)$$

gives a somewhat neater version of equation 2.12

$$P_{l,m,p}^{n+1} = (2 - 6\lambda^2) P_{l,m,p}^n + \lambda^2 S - P_{l,m,p}^{n-1} \quad (2.15)$$

More than simply being neat, this is now a representation of how to calculate new values for the three dimensional array from the previous ones using a simple stencil operation.

2.2 Graphics Processing Units (GPUs)

GPUs were originally designed for 3D game rendering [4]. For this purpose they are equipped with the ability to process large arrays of floating point numbers very efficiently, both in terms of operations per second, and per Watt. It so happens that these

qualities make these devices very well suited to the field of High Performance Computing (HPC). As it turns out, this coincidence is a happy one indeed, as it allows the very high research and development costs of bringing a new processor to market to be spread across the much larger gaming industry. The CEO of NVIDIA, which is the largest supplier of discrete graphics cards, described the budget of several billion US dollars for the design of the latest Pascal GPU architecture as so large it would probably be enough to send someone to Mars [5].

GPUs have a number of technical features which make them particularly suited to running FDTD simulations. GPUs were shown in [1] to perform up to 2 orders of magnitude better than CPUs when running room acoustics simulations.

2.2.1 GPUs and parallelism

GPUs primarily achieve their massive performance increases over CPUs by harnessing operating in a massively parallel fashion. This is done principally with a single instruction multiple data (SIMD) approach. This is where the same processor instructions can execute on multiple objects in memory.

2.2.2 GPU Memory Hierarchies

2.3 The State of the Art in FDTD Acoustical Modelling

Combining the ideas as written about in sections 2.1 and 2.2, it is possible to describe the most successful techniques thus far in running FDTD room acoustics models.

2.3.1 Memory Considerations

The performance of FDTD models can be measured using two factors: the computation time and the amount of memory used. Because of their high bandwidth memory and fast floating point performance, GPUs routinely perform very well in the computation time category. However, the graphics memory that they operate on is much more limited in its supply than standard DRAM. By way of example, the high end HPC targeted NVIDIA K20 device primarily used in this project has 5 GiB. This is significantly less than many modern laptops will have as main memory.

Going back to equation 2.12, it is possible to derive a stability relation for the value of the Courant number, or λ . The approximate solution to the differential equation is said to be stable if results do not grow exponentially. A full derivation is given in [8] using Von Neumann spectral analysis. This yields the important result that for stability it is required that $\lambda \leq \frac{1}{\sqrt{3}}$.

This gives a concrete relation between the sample rates in time ($\frac{1}{T}$) and space (H).

$$H \leq \sqrt{3}cT \quad (2.16)$$

High sample rates in time are required to avoid temporal aliasing. This is an effect where the ear lacks enough information to determine whether a signal is one of two possible values within the audible range (≈ 20 Hz-20 kHz). The Nyquist limit requires that the sample rate be twice as high as the highest audible frequency to avoid this aliasing effect [9]. This requires a very large number of points for even a moderate sized room. Taking a sample rate of 44.1 kHz (as is used in audio CDs), and the speed of sound in air as 344 m s^{-1} the Courant limit becomes 0.0135 m. This means that to model a room of even 1 m^3 a matrix of 75 by 75 by 75 points or 421 875 floating point numbers are required. This means using 3.22 MiB or 1.61 MiB of memory for double or single precision respectively per m^3 .

2.4 Other Methods

The methods outlined in this section are described in a deliberately terse way. They do not form the body of the work done for this dissertation. They are discussed here to offer some justification as to why not, as well as to give the FDTD approach some context within its field.

2.4.1 Ray Tracing

2.4.2 Spectral Methods

$$h(x) = f \otimes g = \int_{-\infty}^{\infty} f(x-u)g(u)du = \mathcal{F}^{-1}\left(\sqrt{2\pi}\mathcal{F}[f] \cdot \mathcal{F}[g]\right) \quad (2.17)$$

Chapter 3

The Proposed Data Structure and Method

“Nothing surpasses the beauty and elegance of a bad idea.”

Craig Bruce

Chapter 4

Design and Implementation

“The first 90 percent of the code accounts for the first 90 percent of the development time...The remaining 10 percent of the code accounts for the other 90 percent of the development time.”

Tom Cargill

4.1 Design Overview

4.2 Software Development Process

4.3 Debugging and Profiling Tools

Chapter 5

Results and Analysis

5.1 Some results

Here are some results.

5.1.1 More results

When showing results you are likely to use tables and graphs. You can create tables easily in Latex.

File names	Satellite	Resolution
worldr	Meteosat	5km
worldg	Meteosat	5km
worldb	Meteosat	5km

Table 5.1: This is a simple table. More complicated tables can have headings which pass over more than one column

Chapter 6

Conclusions

6.1 Future Work



Figure 6.1: The University Crest

Appendix A

NVIDIA K20 Device Details

CUDA Driver Version / Runtime Version	8.0 / 7.0
CUDA Capability Major/Minor version number:	3.5
Total amount of global memory: (5307367424 bytes)	5062 MBytes
(13) Multiprocessors , (192) CUDA Cores/MP:	2496 CUDA Cores
GPU Max Clock rate:	706 MHz (0.71 GHz)
Memory Clock rate:	2600 Mhz
Memory Bus Width:	320-bit
L2 Cache Size:	1310720 bytes
Maximum Texture Dimension Size (x,y,z) =(65536, 65536), 3D=(4096, 4096, 4096)	1D=(65536), 2D
Maximum Layered 1D Texture Size, (num) layers layers	1D=(16384), 2048
Maximum Layered 2D Texture Size, (num) layers 2048 layers	2D=(16384, 16384),
Total amount of constant memory:	65536 bytes
Total amount of shared memory per block:	49152 bytes
Total number of registers available per block:	65536
Warp size:	32
Maximum number of threads per multiprocessor:	2048
Maximum number of threads per block:	1024
Max dimension size of a thread block (x,y,z):	(1024, 1024, 64)
Max dimension size of a grid size (x,y,z):	(2147483647, 65535, 65535)
Maximum memory pitch:	2147483647 bytes
Texture alignment:	512 bytes
Concurrent copy and kernel execution: engine(s)	Yes with 2 copy
Run time limit on kernels:	No
Integrated GPU sharing Host Memory:	No
Support host page-locked memory mapping:	Yes
Alignment requirement for Surfaces:	Yes
Device has ECC support:	Disabled
Device supports Unified Addressing (UVA):	Yes
Device PCI Domain ID / Bus ID / location ID:	0 / 2 / 0

Appendix B

Stuff which no-one will read

Some people include in their thesis a lot of detail, particularly computer code, which no-one will ever read. You should be careful that anything like this you include should contain some element of uniqueness which justifies its inclusion.

Bibliography

- [1] Webb, C.J. (2009) *Parallel computation techniques for virtual acoustics and physical modelling synthesis*. PhD thesis. University of Edinburgh. Available at: http://www2.ph.ed.ac.uk/~cwebb2/Site/Docs/CJWebb_thesis_FINAL.pdf (Accessed: 5 April 2017)
- [2] Hamilton, B. (2016) *Finite Difference and Finite Volume Methods for Wave-based Modelling of Room Acoustics*. PhD thesis. University of Edinburgh. Available at: <http://www2.ph.ed.ac.uk/~s1164563/thesis/hamilton2016phdthesis.pdf> (Accessed: 5 April 2017)
- [3] Richard Feynman, 1969, *Lectures in Physics, Volume 1*, Addison Publishing Company, Addison
- [4] NVIDIA Corporation *What's the Difference Between a CPU and a GPU?* <https://blogs.nvidia.com/blog/2009/12/16/whats-the-difference-between-a-cpu-and-a-gpu/> retrieved July 17, 2017.
- [5] NVIDIA Corporation *NVIDIA Special Event: The GeForce GTX 1080 and Pascal (Part 4)* <https://www.youtube.com/watch?v=0xnzwjPyx8A> retrieved July 17, 2017.
- [6] P. Micikevicius, *3D finite difference computation on GPUs using CUDA*. Proceedings of 2nd Workshop on General Purpose Processing on Graphics Processing Units - GPGPU-2, New York, New York, USA, 2009, pp. 79-84, ACM Press.
- [7] K. Hou, Y. Zhao, J. Huang, and L. Zhang, *Performance evaluation of the Three-Dimensional Finite-Difference Time-Domain(FDTD) method on Fermi architecture GPUs in Algorithms and Architectures for Parallel Processing* vol. 7016 of Lecture Notes in Computer Science, pp. 460-469. Springer Berlin / Heidelberg, 2011.
- [8] J. Strikwerda, *Finite Difference Schemes and Partial Differential Equations*, Wadsworth and Brooks/Cole Advanced Books and Software, Pacific Grove, California, 1989.
- [9] Harris, Frederic j. (Aug 2006). *Multirate Signal Processing for Communication Systems*. Upper Saddle River, NJ: Prentice Hall PTR. ISBN 0-13-146511-2

- [10] L.Lamport. *1986 Latex User's Guide and Reference Manual*. Addison Wesley.
pp242.
- [11] F.Bloggs. *1993 Latex Users do it in Environments* Int. Journal of Silly Findings.
pp 23-29.